PRACTICAL WEEK-

11 SESSION-11:

Task-1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_ioin). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Source code:

```
  GNU nano 8.7.1                  thread_counter.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    int i;

    for (i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("===== MULTITHREADED COUNTER =====\n");
    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Increments/thread : %d\n\n", INCREMENTS);

    /* Create threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                           increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
```

```
    /* Wait for all threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value : %d\n",
           NUM_THREADS * INCREMENTS);

    printf("Actual counter value   : %d\n", counter);

    if (counter == NUM_THREADS * INCREMENTS)
        printf("No race condition observed.\n");
    else
        printf("Race condition detected!\n");

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$ nano thread_counter.c
sathwika@DESKTOP-AA8K5MG:~$ gcc thread_counter.c -o thread_counter -pthread
sathwika@DESKTOP-AA8K5MG:~$ ./thread_counter
===== MULTITHREADED COUNTER =====
Number of threads : 4
Increments/thread : 100000

Expected counter value : 400000
Actual counter value   : 147109
Race condition detected!
sathwika@DESKTOP-AA8K5MG: ~$
```

Observation:

The multithreaded counter application was successfully implemented using POSIX threads. The experiment demonstrated that **concurrent updates to a shared variable without synchronization can cause a race condition and produce an incorrect counter value**.

Task-2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

Source code:

```
  GNU nano 8.7.1                  mutex_counter.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

int counter = 0;

/* Mutex declaration */
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    int i;

    for (i = 0; i < INCREMENTS; i++)
    {
        /* Lock before accessing shared variable */
        pthread_mutex_lock(&lock);

        counter++;

        /* Unlock after updating shared variable */
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("===== MUTEX SYNCHRONIZATION =====\n");
    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Increments/thread : %d\n\n", INCREMENTS);

    /* Initialize mutex */
    pthread_mutex_init(&lock, NULL);
```

```c
    /* Create threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                       increment_counter, NULL);
    }

    /* Wait for all threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value : %d\n",
           NUM_THREADS * INCREMENTS);

    printf("Actual counter value   : %d\n", counter);

    /* Destroy mutex */
    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$ nano mutex_counter.c
sathwika@DESKTOP-AA8K5MG:~$ gcc mutex_counter.c  -o mutex_counter -pthread
sathwika@DESKTOP-AA8K5MG:~$ ./mutex_counter
===== MUTEX SYNCHRONIZATION =====
Number of threads : 4
Increments/thread : 100000

Expected counter value : 400000
Actual counter value   : 400000
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

The program was successfully modified using POSIX mutex locks. **The race-condition version can produce incorrect results, while the mutex version produces the correct result by protecting the shared counter.** The experiment demonstrates that synchronization improves data consistency at the cost of some additional execution overhead.